Short test 1 — practice tasks

Interactive Theorem Proving using Lean · 11 November 2026

The short test on 11 November has six tasks of the same kinds as these twelve. In the test, each task has three parts worth one point each; the test is passed with 10 of the 18 points. The answers are on the last pages. 20 minutes, no computer, no notes. In all tasks, `P Q : Prop`.

Scope: chapters 0, 1.1 and 1.2, and the sheets `L-a` to `L-d`, `1.1-a`, `1.1-b` and `1.2-a`.

Task 1. The proof state is

```
⊢ ∀ n : ℕ, ∃ m : ℕ, m > n
```

Write down the proof state after `intro k`.

Task 2. The proof state is

```
R : ℕ → Prop
h : ∀ n : ℕ, R (n + 1)
⊢ ∃ m : ℕ, R m
```

Write down the proof state after `use 5`.

Task 3. The proof state is

```
⊢ P ↔ Q
```

Write down the proof state after `constructor`.

Task 4. The proof state is

```
h : P ∧ Q
⊢ Q ∧ P
```

Write down the proof state after `obtain ⟨hp, hq⟩ := h`.

Task 5. The proof state is

```
hp : P
hnp : ¬P
⊢ Q
```

Write down the proof state after `exfalso`.

Task 6. The proof state is

```
hnp : ¬P
hp : P
⊢ False
```

Write down the proof state after `apply hnp`.

Task 7. Give the type of each term.

term	type
`Prop`	
`Type 2`	
`∀ n : ℕ, n = n`	

Task 8. Give the type of each term.

term	type
`fun n : ℕ => (0 : Fin (n + 1))`	
`Nat.succ`	
`some 3`	

Task 9. Name the reduction rule (β, δ, ι or ζ) of each step. Here `myDouble n` is defined as `Nat.rec 0 (fun _ r => r + 2) n`.

step	rule
`(fun x => x + 1) 4 → 4 + 1`	
`myDouble 2 → Nat.rec 0 (fun _ r => r + 2) 2`	
`Bool.rec 5 7 true → 7`	
`let x := 2; x + x → 2 + 2`	

Task 10. Here `n : ℕ` and `b : Bool`. Tick every statement that `rfl` proves.

- ☐ `n + 1 = Nat.succ n`
- ☐ `1 + n = Nat.succ n`

- ☐ `(fun x => n + x) 0 = n`
- ☐ `(fun x => x + n) 0 = n`
- ☐ `(false && b) = false`
- ☐ `(b || true) = true`

Task 11. What is the value of

```
Nat.rec 0 (fun n r => r + 2 * n + 1) 4 : ℕ
```

Answer with a number.

Task 12. For each of the three questions about a statement in Lean, write *decidable*, *semi-decidable* or *undecidable*.

question	answer
Does this text prove that statement?	
Find a proof of this statement.	
Is this statement provable, yes or no?	

Answers

Case names (`mp`, `inl`, `h`) and the line `P Q : Prop` are not required.

Task 1.

```
k : ℕ
⊢ ∃ m : ℕ, m > k
```

Wrong: `n` instead of `k`, or the `∃` removed as well.

Task 2.

```
R : ℕ → Prop
h : ∀ n : ℕ, R (n + 1)
⊢ R 5
```

`h` stays. Wrong: `h` changed, or `R 6`, or no goal. (`use` tries to close the goal by `rfl` or an assumption; here neither works.)

Task 3. Two goals:

```
⊢ P → Q
⊢ Q → P
```

Both needed, in either order. Wrong: `P` and `Q` as goals.

Task 4. One goal:

```
hp : P
hq : Q
⊢ Q ∧ P
```

Wrong: two goals, or `h` still present.

Task 5.

```
hp : P
hnp : ¬P
⊢ False
```

The hypotheses stay. Wrong: anything but `False` as the goal.

Task 6.

```
hnp : ¬P
hp : P
⊢ P
```

`hnp : P → False`, so applying it to the goal `False` leaves `P`. Wrong: `⊢ ¬P` or no goal.

Task 7. `Type`, `Type 3`, `Prop`. All three needed. The third: a `∀` over `ℕ` is still a proposition.

Task 8. `(n : ℕ) → Fin (n + 1)`, `ℕ → ℕ`, `Option ℕ`. All three needed. For the first, `ℕ → Fin (n + 1)` is wrong (the `n` must be bound); `∀ n : ℕ, Fin (n + 1)` is fine.

Task 9. β, δ, ι, ζ. All four needed. The third also tests the order of the cases of `Bool.rec`: the `false` case comes first.

Task 10. Exactly 1, 3 and 5. After β, `(fun x => x + n) 0` is `0 + n`; `+` recurses on the second argument, so `0 + n` and `1 + n` are stuck at `n`. `||` recurses on the first argument, so `b || true` is stuck at `b`.

Task 11. `16`. The values at `0, 1, 2, 3, 4` are `0, 1, 4, 9, 16`: the step adds the odd number `2n + 1`, so the recursor computes `n²`.

Task 12. Decidable, semi-decidable, undecidable. All three needed. See chapter 0, “Checking is not finding”: the kernel checks; a search finds a proof when there is one and may run forever when there is none; whether a proof exists is undecidable (Church, Turing 1936).